

SUMO24 MCP Tools Reference

235 tools exposed by the SUMO24 MCP server
Generated from the live server tool list — revision v11

This reference catalogues every tool the SUMO24 MCP server exposes to Claude. Tools are grouped functionally — core simulation control at the top, then diagnostics, scenarios, parameters, datasets, validation, optimisation, academic exports, the WWTP Excel template pipeline (Group AA), the HTML schematic builder pipeline (Group BB), the SUMO file troubleshooting package (Group CC), the schematic-to-SUMO compiler pipeline (Group DD), the guided modelling pipeline (Group EE), the DTT edit-transaction batch layer (Group FF), the SUMO GUI automation surface (Group GG), and the SumoSlang unit-authoring toolchain (Group HH). All tools are callable through the MCP protocol; each accepts a JSON object matching its inputSchema and returns a JSON-encoded TextContent.

Group	Name	Tool count
Group B	Simulation Runs	2
Group C	Reports & Diagnostics	6
Group D	Topology Editing	19
Group E	Model Lifecycle	11
Group F	Scenarios	8
Group G	Parameters	26
Group H	Datasets	17
Group X	Validation	16
Group Y	Optimisation / Diagnostics / Economics	13
Group Z	Academic Export	18
Group AA	WWTP Excel Template	5
Group BB	HTML Schematic → SUMO	6
Group CC	SUMO Troubleshooting	7
Group DD	Schematic-to-SUMO Compiler	12
Group EE	Modelling Pipeline	19
Group FF	DTT Edit & Transactions	5
Group GG	SUMO GUI Automation	21
Group HH	SumoSlang Unit Authoring	24
TOTAL		235

Group B — Simulation Runs

2 tools

Tool	Description
run_steady_state	Run a SUMO24 steady-state simulation of the Qaha WWTP model. Optionally override influent parameters before running. Returns effluent quality summary and Law 48/1982 compliance status.
run_dynamic_simulation	Run a SUMO24 dynamic simulation for a specified number of days. Useful for evaluating transient behaviour, storm events, or seasonal loads.

Group C — Reports & Diagnostics

6 tools

Tool	Description
get_job_status	Check the status of a previously scheduled simulation job.
get_effluent_statistics	Return min/max/mean for all numeric columns in the CSV of a completed job.
get_sludge_production	Estimate WAS rate, SRT, and biosolids production from a completed job (best-effort; notes what to look up).
get_energy_estimate	Pull DO setpoints and airflow-related parameter values for a scenario as a proxy for aeration energy.
list_completed_jobs	Return every job currently in memory with status, scenario, and type (steady / dynamic).
get_mass_balance	Compute COD / N / P / TSS mass balance (influent vs effluent) for a completed job from its exported CSV.

Group D — Topology Editing

19 tools

Tool	Description
list_unit_processes	List all unit process names currently visible in the compiled model.
get_unit_process_info	Return parameters and state variables for a specific unit process.
list_available_unit_processes_types	Return all unit process types supported by SUMO24 (hydraulic, biological, clarification, tertiary, sludge, sensors, controllers).
add_unit_process	Add a treatment unit process to the active model via DTT executeCommand; optionally assigns a kinetic model.
remove_unit_process	Remove a unit process from the active model and optionally all streams connected to it.
rename_unit_process	Rename a unit process in the loaded .sumo model via DTT executeCommand. References to the old name in streams and controllers are updated by SUMO; the live model is modified in memory. Persist with save_model when done.
change_unit_type	Change the class of an existing unit (e.g. Generic PrimaryClarifier -> Otterpohl) via DTT. Use with care: incompatible class swaps will be rejected by the solver. dry_run returns the planned DTT command without dispatching.

list_available_kinetic_models	List biological kinetic models available for assignment to reactors (ASM1, ASM2d, ASM3, ADM1, MBR, Takács, etc.).
connect_unit_processes	Create a flow connection (stream) between two unit processes.
add_recycle_stream	Add a recycle stream (RAS, WAS, internal recycle, reject water, or sidestream) between two units.
modify_flow_connection	Redirect or resize a flow stream (change source, destination, or fixed flow rate).
remove_flow_connection	Delete a stream from the loaded model via DTT. The complement to <code>modify_flow_connection</code> / <code>connect_unit_processes</code> . <code>dry_run</code> returns the planned command.
set_stream_flow_rate	Set a fixed flow rate (m3/d) on an existing stream via DTT — used for RAS/WAS/IR loops or any fixed-Q stream.
list_flow_streams	List registered flow streams and compiled model flow variables, optionally filtered to one unit.
add_controller	Add a control loop (DO, SRT, NH4, NO3, FlowSplitter controller) to a unit with sensor/actuator and setpoint.
list_controllers	List controllers (DO / SRT / NH4 / NO3 / FlowSplitter) in the loaded model. Tries DTT enumeration first; falls back to the manifest.xml of the on-disk .sumo if DTT doesn't expose a list API.
modify_controller	Modify an existing controller via DTT — setpoint, gain, integral time, output min/max. Any field left null is left unchanged. Use <code>list_controllers</code> first to confirm the controller id.
remove_controller	Remove a control loop from the loaded model via DTT. Complement to <code>add_controller</code> .
ensure_dtt_bridge	Detect and (re)install the dynamita_compat shim that backfills <code>SumoScheduler.executeCommand/.set/.getVariableNames</code> when the installed Dynamita DTT distribution doesn't expose them. Returns patch status and the current queue of buffered commands. Optionally flushes the queue into a sibling <code><project>.commands.txt</code> next to the given .sumo path.

Group E — Model Lifecycle

11 tools

Tool	Description
get_model_info	Return top-level model metadata: DLL path, state.xml, total variables, parameter count, compile status.
save_state	Export the current in-memory model state to a state.xml snapshot (default: timestamped file in outputs/).
load_state	Point the server at a different state.xml snapshot for all subsequent runs.
list_saved_states	List state.xml snapshots found in the outputs directory.
get_compile_status	Report whether the project DLL and state.xml exist and are ready to run, with modification timestamps.
create_model	Initialise a new blank SUMO24 project folder with a .sumo stub file, subdirectories, and README. Use before building a WWTP from scratch.
load_model	Load an existing .sumo project file and register it as the active model; auto-detects adjacent DLL and state.xml.

save_model	Save the current project to disk: copies .sumo + DLL, and saves a fresh state.xml snapshot via DTT.
compile_model	Check whether a runnable DLL exists or force a recompile by backing up and removing the current DLL (actual compilation happens in SUMO GUI).
extract_dll	Locate and register a compiled sumoproject.dll. Accepts: (a) an explicit .dll path; (b) a .sumo project file — in which case sumoproject.dll is unzipped from the archive alongside it (a stale sibling DLL is backed up first); (c) omitted — scans the project dir and cwd.
initialize_state	Reset the model to its baseline state.xml â€” clears all in-memory parameter overrides.

Group F — Scenarios

8 tools

Tool	Description
run_scenario_comparison	Run multiple named scenarios sequentially and return a side-by-side compliance comparison table (TSS, BOD5, COD vs Law 48/1982 limits).
list_scenarios	List all available pre-defined Qaha WWTP scenarios with their parameter changes.
create_scenario	Add a new named scenario to the in-memory SCENARIOS dict.
update_scenario	Replace the parameter overrides of an existing scenario.
delete_scenario	Remove a scenario from the in-memory SCENARIOS dict.
clone_scenario	Duplicate an existing scenario under a new name.
get_scenario_diff	Show which parameters differ between two scenarios.
import_scenario	Load a .scs script file and register it as a scenario.

Group G — Parameters

26 tools

Tool	Description
set_parameter	Set any SUMO24 model parameter by its full SumoCore variable name. Changes are applied on the next simulation run (not permanently saved). Use the SumoCore Advanced Core Window to find variable names.
get_parameter	Read the current value of a single model parameter (optionally after applying a scenario).
list_parameters	List configurable parameters (__param__ variables). Optional substring filter on unit name.
set_multiple_parameters	Set several model parameters in one call (in-memory; applies to next simulation run).
reset_parameter_to_default	Revert an in-memory parameter to the value stored in state.xml.
set_kinetic_model	Assign or switch the biological kinetic model on a reactor unit.

set_influent_characteristics	Define influent wastewater composition and flow rate in one call; maps lab parameters (BOD, COD, TSS, TKN, NH4, TN, TP, temp) to their SUMO variables.
set_fractionation	Set COD/TSS fractionation ratios (Xs, Xp, Si, Ss, Xi, fXS, etc.) on a unit for ASM-type kinetic models.
set_parameter_persistent	Persist a parameter change directly into state.xml on disk (with timestamped backup) so it survives server restarts.
set_asm1_kinetics	Set ASM1 kinetic + stoichiometric parameters on a biological unit. Any param left null uses the IWA/Egyptian research default, Arrhenius-corrected to temp_C.
set_asm2d_kinetics	Set ASM2d kinetic + stoichiometric parameters (heterotrophs, PAOs, autotrophs, precipitation). Any param left null uses the research default at temp_C.
set_asm3_kinetics	Set ASM3 kinetic + stoichiometric parameters (endogenous respiration + internal storage X_STO). Any param left null uses the research default at temp_C.
apply_temperature_correction	Re-compute Arrhenius temperature-corrected ASM parameters at a new operating temperature. Reports per-parameter % change and flags high-impact (>20%) corrections.
set_plant_wide_parameters	Set the global plant temperature and update CONFIG Law 48 limits to match the chosen discharge target ('nile' or 'drain').
set_influent_parameters	Set influent flow, composition, and COD fractionation. Defaults to typical Egyptian municipal wastewater values if arguments are omitted.
set_primary_clarifier_parameters	Set primary clarifier design parameters (area, overflow rate, solids loading, capture efficiency, HRT). Defaults from Metcalf & Eddy 5th ed.
set_aeration_tank_parameters	Set aeration-tank hydraulic / process / aeration parameters (V, HRT, SRT, DO, KLa, SOTE, MLSS, T). Defaults tuned for Egyptian warm-climate operation.
set_anoxic_zone_parameters	Set anoxic zone parameters (volume, HRT, NO3 recycle ratio, max DO). Defaults for pre-denitrification.
set_anaerobic_zone_parameters	Set anaerobic (EBPR / P-release) zone parameters (volume, HRT, max DO, VFA target).
set_secondary_clarifier_parameters	Set secondary clarifier hydraulic + Takacs settler model parameters (v0, v0_p, r_h, r_p, f_ns). Defaults from Takacs 1991 / BSM1.
set_mbr_parameters	Set membrane bioreactor parameters (flux, TMP, permeability, backwash interval/duration, MLSS). Defaults from Judd MBR Book / Metcalf & Eddy.
set_ras_was_parameters	Set RAS + WAS pump parameters (flows, ratios, target SRT). Calculates Q_RAS from RAS_ratio if flow is omitted.
set_digester_parameters	Set aerobic or anaerobic digester parameters (V, SRT, T, DO, OLR, biogas yield, VSS reduction).
set_chemical_dosing_parameters	Set chemical precipitation / coagulant dosing parameters (alum or ferric). Estimates dose from target TP if not supplied.
set_dynamic_simulation_parameters	Configure the SUMO24 dynamic simulation tab (duration, timestep, solver, warm-up, diurnal and storm factors, seasonal temperatures).

assume_parameters_from_research	Auto-populate a full recommendation of research-based defaults (influent, ASM kinetics, aeration tank, clarifier, dynamic simulation) sized to the selected plant_scale and temperature. Does NOT apply values to the model - returns a recommendation dict to review first.
--	--

Group H — Datasets

17 tools

Tool	Description
create_dynamic_input_table	Build a time-series input table for a specific SUMO24 parameter. Automatically reads the current simulation period (Sumo__StopTime) and data interval (Sumo__DataComm) from state.xml so the generated table matches the simulation settings exactly. The user can also override the start/end time step (range) and the row interval. Supports constant, linear ramp, step change, sinusoidal, and custom value profiles. Writes a TSV file under outputs/ ready to be passed to run_dynamic_simulation via influent_tsv.
list_state_variables	List model state variables (non-parameter variables). Optional substring filter.
get_state_variable	Read the current value of a specific state variable.
get_state_variables	Bulk read of many state variables in a single tool call. Returns {values: {var: value_or_null}, missing: [...]}. Use this instead of N separate get_state_variable calls — it collapses N round-trips into 1 and works against the offline fallback (laststeadyrun.ss / lastdynamicrun.ss) when DTT is not live.
extract_sumo_data	Offline one-shot extractor for a .sumo project archive. Returns the full bundle an academic report needs: design parameters, dynamic influent statistics, steady-state and dynamic-final effluent quality (per clarifier), bioreactor MLSS / DO per CSTR zone, and biomass composition. Reads the zip directly — no SUMO24 GUI / DTT required. Replaces dozens of separate get_state_variable / PowerShell calls.
search_variables	Case-insensitive substring search across all variable and parameter names.
import_influent_profile	Register a time-varying influent CSV so run_dynamic_simulation can reference it by label.
read_dataset_excel	Read the WWTP Excel dataset template and return a structured summary of all measurement points, Law 48 limits, and section headers.
list_dataset_measurement_points	List every measurement point in the dataset and the SUMO unit it maps to.
get_measurement_point_data	Get all measured parameters for a specific measurement point (e.g. 'Raw Influent'). Flags Law 48 exceedances.
get_parameter_across_points	Trace a single parameter (e.g. COD) across every measurement point. Reports overall influent-to-effluent removal pct.
get_law48_limits_from_excel	Return the Law 48/1982 limit row from the Excel template and sync into CONFIG['law48_limits_from_dataset'].
map_dataset_to_sumo_variables	Resolve every dataset cell (point x parameter) to a concrete SUMO variable in the compiled model. Reports resolved + unresolved.
validate_dataset	Sanity-check the dataset: missing critical parameters, physical ranges (pH, Temp, DO), COD trend, flow continuity, Law 48 exceedances.
check_dataset_against_law48	Compare every measurement point's values against the Law 48 limit row; flags exceedances stage-by-stage.

apply_influent_from_dataset	Apply one row (default 'Raw Influent') from the Excel dataset to the corresponding SUMO Influent unit parameters.
apply_point_to_unit	Apply a specific measurement point's values to a SUMO unit (supports unit_override for custom unit names).

Group X — Validation

16 tools

Tool	Description
validate_variable_name	Check whether a SumoCore variable name exists in the compiled model. If not, returns up to 10 name suggestions.
validate_model	Run configuration + readiness checks on the current model. 'full' mode also runs a quick steady-state to surface solver errors.
validate_model_structure	Structural pre-flight check: verifies DLL + state.xml exist, are version-consistent, and model has required unit types (influent, reactor, clarifier, effluent).
validate_influent_configuration	Check that the influent is configured with sensible, non-zero values (flow > 0, COD > 0, temperature and pH within biological range, COD:TKN ratio).
validate_mass_balance_pre_run	Quick pre-simulation mass balance sanity check. Compares influent load to state.xml effluent values to flag obvious configuration errors for COD, TSS, TKN, TP.
validate_mass_balance_post_run	Full mass balance validation on a completed simulation. For each element (COD, N, P, TSS) computes closure error = (in - out - WAS) / in. Well-behaved plants close within +/- 5%.
validate_mlss_health	Check MLSS, MLVSS/MLSS ratio, active heterotroph and nitrifier fractions in a reactor against Metcalf & Eddy ranges.
validate_srt_fm_feasibility	Check whether configured SRT and F/M are feasible. Computes actual SRT from V, MLSS, Q_WAS, X_R and compares to target; computes F/M and HRT.
validate_do_levels	Verify DO levels are within correct range for each zone type (aerobic 1.5-2.5, anoxic ~0, anaerobic ~0). Also checks KLa.
validate_oxygen_demand_vs_supply	Check aeration supply sufficiency. Estimates O2 demand from BOD + nitrification and compares with KLa-driven supply.
validate_hydraulic_balance	Validate flow continuity ($Q_{in} \sim Q_{eff} + Q_{WAS}$), RAS ratio, secondary clarifier SOR, and underflow concentration.
validate_asm_kinetics	Check ASM kinetic parameters (μ_H , K_S , b_H , Y_H , μ_A , K_{NH} , b_A , Y_A , k_h , K_X) against IWA/Henze physical bounds. Catches typos like $Y_H = 6.7$.
validate_simulation_convergence	Check if a simulation reached steady state or is dynamically stable. Analyzes the last N% of the time series for key effluent parameters.
validate_simulation_plausibility	Sanity-check simulation output for physically impossible values: negative concentrations, pH outside 4-10, oversaturated DO, effluent > influent for conserved substances.
validate_full_model	Run every pre-simulation validation in sequence. Returns READY_TO_SIMULATE / CHECK_WARNINGS / DO_NOT_SIMULATE plus an aggregated report. Use as the gate before any run_call.

validate_post_simulation	Run every post-simulation validation (mass balance, convergence, plausibility) and return RESULTS_VALID / RESULTS_SUSPECT / RESULTS_INVALID.
---------------------------------	--

Group Y — Optimisation / Diagnostics / Economics

13 tools

Tool	Description
check_compliance	Given a simulation result (job_id or direct values), evaluate compliance with Egyptian Law 48/1982 effluent limits.
apply_dataset_to_model	Apply ALL numeric values from the dataset to the SUMO model in one call (full calibration import). Supports dry_run preview and only_influent mode.
compare_dataset_vs_simulation	Compare measured values from the Excel dataset against simulated values from a completed job. Reports per-point deltas and Mean Absolute % Error (MAPE).
compute_removal_efficiencies_from_dataset	Compute per-stage and overall removal efficiencies for BOD5, COD, TSS, NH4, TN, TP from Excel data alone.
scan_parameter_sensitivity	Sweep a parameter across a list of values and record an output variable at each point. Instantaneous reads - pair with run_steady_state for full effluent sensitivity.
optimize_was_for_target_srt	Find and apply the Q_WAS that achieves a target SRT. Uses analytical $SRT = V \cdot MLSS / (Q \cdot WAS \cdot X_R)$ to solve directly for Q_WAS.
optimize_kla_for_target_do	Estimate and apply the KLa needed to hit a target DO setpoint in the aerobic zone. Analytical solution from $OTR = KLa \cdot V \cdot (DO_{sat} - DO)$.
diagnose_nitrification_failure	Targeted diagnostic for failed nitrification. Checks SRT, temperature, pH, DO, and effluent NH4, then returns likely root causes + recommended fixes.
diagnose_bulking_risk	Sludge bulking risk assessment. Evaluates SVI, F/M ratio, DO, and COD:TP to flag filamentous/viscous bulking conditions.
diagnose_phosphorus_removal	EBPR diagnostic. Checks anaerobic DO, NO3 intrusion, COD:TP ratio, and effluent P to identify why biological phosphorus removal is failing.
recommend_setpoint_adjustments	Aggregate diagnostic + recommended operational adjustments. Ranks recommendations by priority (critical/high/medium/none) with the tool to use for each fix.
estimate_annual_opex	Annual OPEX estimate: energy (aeration + pumping), sludge disposal, and chemicals. Returns USD/year and breakdown.
estimate_ghg_emissions	Annual GHG emissions (kg CO2eq/year) = Scope 2 electricity + Scope 1 direct N2O (0.5% of TKN load) + Scope 1 CH4 from digesters.

Group Z — Academic Export

18 tools

Tool	Description
export_results_csv	Export the time-series results of a completed simulation job to a CSV file.
export_scenario	Save a scenario to a .scs script file.

generate_report	Produce a structured design summary (effluent, compliance, metadata) for a completed job.
export_simulation_to_dataset	Create an Excel file (copy of template) with current simulation values filled into every measurement point. Original file is not modified.
export_comparison_report	Create an Excel calibration report with Measured / Simulated / Delta / Delta-pct / Fit-Quality columns on a 'Calibration Comparison' sheet, color-coded.
plot_time_series_chart	Generate a PNG time-series chart from a completed job's CSV for selected effluent variables.
export_publication_time_series	Export publication-quality time-series plot (serif fonts, 300 DPI, colorblind palette, PNG+PDF+SVG). Supports multiple simulation job_ids and multiple variables overlaid.
export_calibration_parity_plot	Export a calibration/validation 1:1 parity plot (simulated vs observed) with R2, NSE, RMSE, PBIAS statistics annotated. Publication-quality styling.
export_scenario_comparison_bar_chart	Grouped bar chart (mean +/- SD error bars) comparing multiple effluent variables across multiple simulation jobs. Academic styling, multi-format output.
export_sensitivity_tornado_diagram	Publication-quality tornado diagram showing parameter sensitivity. Ranks parameters by range of output deviation from baseline.
export_box_plot_distributions	Box-plot (or violin-plot) comparing the distribution of a single variable across multiple simulation jobs. Includes per-job descriptive statistics.
export_academic_results_table_docx	Export a Word (.docx) table with publication/booktabs-style formatting (Times New Roman, heavy top/bottom rules, italic units row). Suitable for direct inclusion in a journal manuscript.
export_academic_bundle	One-shot academic export for a .sumo project. Generates the 4 booktabs Word tables (influent, effluent + Law 48 compliance, bioreactor, removal), 4 publication figures (PNG + PDF), a 6-sheet Excel workbook, and a combined CSV — all from a single .sumo file, with no SUMO24 GUI / DTT required. Reads laststeadyrun.ss + lastdynamicrun.ss + parameters.txt + Influent TSVs directly. Replaces dozens of separate get_state_variable + export_* calls.
export_latex_results_table	Export a LaTeX booktabs table (toprule/midrule/bottomrule) as a .tex file, ready to include in a journal manuscript. Includes auto-escaping of special characters.
export_statistical_summary_xlsx	Export a multi-sheet Excel workbook with descriptive statistics (n/mean/SD/CV%/min/p05/p25/median/p75/p95/max) for multiple simulation jobs. One sheet per job plus Combined_Mean and Combined_Std sheets.
export_academic_report_docx	Generate a full academic Word report (Methods/Results/Discussion/References) with embedded publication-quality figures and booktabs result tables across multiple simulation jobs.
export_multi_file_batch_comparison	Batch export across multiple SUMO files and multiple simulation jobs. Accepts either a list of job_ids or a mapping {sumo_file_label: [job_id, ...]}. Produces combined plots (time-series + scenario bars), a multi-sheet XLSX summary, and a LaTeX booktabs table in one call.
export_mass_balance_diagram	Publication-style mass-balance block diagram showing boxes for unit processes and arrows for flows (arrow thickness scaled by flow magnitude).

Group AA — WWTP Excel Template

5 tools

Tool	Description
generate_wwtp_template.xlsx	Generate a fresh WWTP_Model_Input_Template.xlsx with sheets for plant info, influent/effluent datasets, COD/N/P fractionation, treatment processes (physical, biological, chemical), connections, sizing, and operational parameters. Filled-in templates can be consumed by build_model_from_template.
read_wwtp_template	Parse a filled-in WWTP_Model_Input_Template.xlsx and return all rows (plant info, influent/effluent time series, fractionation, processes, connections, sizing, operational parameters, chemical dosing, effluent targets, and influent statistics).
validate_wwtp_template	Validate a filled-in WWTP template: required Plant_Info fields, duplicate instance names, dangling connections, missing reactor/clarifier sizing, and COD-fraction sum sanity.
build_model_from_template	Build a SUMO24 model from scratch using a filled-in WWTP template. Creates the project folder + .sumo stub, adds every unit process in topological order (sorted by order_index), wires the streams (forward + recycle), applies sizing and operational parameters, sets influent characteristics (mean dataset values + COD/N/P fractionation), and dispatches each step via the DTT executeCommand bridge with a dry_run mode that returns the build plan without touching DTT.
apply_template_auto_fixes	Apply auto_fix suggestions returned by validate_wwtp_template to the template Excel workbook in place. Accepts the list of auto_fix dicts from validate_wwtp_template's error entries and rewrites the workbook. Handles: add_to_treatment_processes (adds WAS/RAS boundary units), clear_na_rows (removes NA filler rows in the Connections sheet). Always makes a .bak copy before writing.

Group BB — HTML Schematic → SUMO

6 tools

Tool	Description
import_schematic_from_html	Read a WWTP schematic file produced by the schematic builder (wwtp_schematic_template.html). Extracts the embedded <script id='schematic-data'> JSON block (or loads a standalone .json export), validates against the unit-type registry, and returns the parsed schematic + validation report. The user must click 'Save HTML (for MCP)' in the builder for the file to contain meaningful state.
build_model_from_schematic	Convert a parsed schematic into SumoCore parameter override commands. Optionally saves the commands to a .scs script and/or applies them to the active compiled SUMO model. By default does NOT create new unit topology — only overrides parameters on units that already exist in the compiled .sumo. Use generate_sumoslang_from_schematic for full topology generation (advanced).
compare_schematic_to_model	Diff a schematic against the currently loaded compiled SUMO model and report which unit IDs exist in both, which are missing in the model, and how many parameter overrides the schematic would apply.
generate_sumoslang_from_schematic	(Advanced / stretch goal) Render a SumoSlang plant-definition skeleton from the schematic. The output is a STARTING POINT only — kinetic constants and class-specific blocks still need hand-editing before SUMO will accept it. Returns the SumoSlang text and optionally writes it to a file.

list_schematic_unit_types	Return the full unit-type registry the schematic parser recognises (mirrors the JS UNIT_TYPES in the HTML builder).
update_schematic_in_html	Safely replace the embedded schematic JSON inside a schematic-builder HTML file. Uses a comment-aware scanner so it never matches a reference inside <!-- ... --> and destroys the file (the bug that hit the Qaha1 session). Validates JSON before writing, validates structural tags after writing, and aborts if either check fails. Creates a .bak backup by default.

Group CC — SUMO Troubleshooting

7 tools

Tool	Description
diagnose_sumo_file	Diagnose a .sumo project file: confirm it exists, that it's a valid zip/XML container, that it contains a sumoproject.dll companion (or report which is missing), checks emptiness (0-byte / truncated), parses the manifest for unit count, lists referenced state.xml, and reports common error patterns.
scan_sumo_directory	Scan a directory recursively for .sumo files and run diagnose_sumo_file on every one. Returns a per-file health report.
check_dll_companion	Verify that sumoproject.dll exists next to a .sumo file (DTT requirement). If missing, attempts to locate the DLL elsewhere in the same project tree and reports a recommended fix.
repair_sumo_file	Attempt automated fixes for the most common .sumo problems: (1) empty/missing state.xml stub, (2) missing sumoproject.dll companion (copy from a known-good source if available), (3) zero-byte .sumo file (recreate a minimal valid stub), (4) wrong file extension. Always backs up the original to <name>.sumo.bak before writing.
diagnose_sumo_crash	Parse the most recent SUMO / DTT log files for stack traces and known error signatures (license errors, DLL load failures, compile failures, etc.) and return a structured diagnosis with suggested fixes.
validate_sumo_environment	End-to-end environment health check for the SUMO24 MCP server: DTT import, sumoproject.dll present, state.xml present, license file present, openpyxl available, schematic_parser available, and the expected directory layout.
list_sumo_diagnostics	Return the full catalogue of known SUMO error signatures and their recommended remediations.

Group DD — Schematic-to-SUMO Compiler

12 tools

Tool	Description
compile_schematic_to_sumo	[DEPRECATED — always returns ok=false. Use build_sumo_pack instead.] Formerly compiled a schematic to a zip .sumo file. That zip format cannot be loaded by SUMO24. This tool now refuses unconditionally and redirects to build_sumo_pack.
build_sumo_from_html	[DEPRECATED — always returns ok=false. Use build_sumo_pack instead.] Formerly built a .sumo zip from an HTML schematic. That zip format cannot be loaded by SUMO24. This tool now refuses unconditionally and redirects to build_sumo_pack.

preview_sumo_manifest	Render and return (without writing) the manifest.xml, state.xml, metadata.json and optional sumoslang.txt that compile_schematic_to_sumo would produce. Useful for inspection or version-control review.
attach_companion_dll	[DEPRECATED — always returns ok=false. Use verify_dll_matches_schematic instead.] Formerly copied a sumoproject.dll next to a .sumo file. Copying a DLL from a different plant produces a symbol-table mismatch and the project will not simulate. This tool now refuses unconditionally.
verify_sumo_file_against_schematic	[DEPRECATED — always returns ok=false. Use verify_dll_matches_schematic instead.] Formerly verified the custom zip format against a schematic. That zip format is not a valid SUMO project. This tool now refuses unconditionally and redirects to verify_dll_matches_schematic.
read_sumo_manifest	Open a .sumo zip file and return a parsed summary of its manifest: unit count, connection count, IDs, metadata, and number of state variables.
build_sumo_pack	Generate a build-pack directory from a schematic. The pack contains plant.sumoslang, apply_parameters.scs, BUILD_INSTRUCTIONS.md, and schematic.json. Open BUILD_INSTRUCTIONS.md and follow the 5-step procedure inside SUMO24 GUI to obtain a working .sumo project. This replaces the broken compile_schematic_to_sumo / build_sumo_from_html pair, which produced zip files SUMO24 could not load.
apply_schematic_to_baseline	Tier-1 path that DOES produce a working .sumo project: copy a verified baseline (.sumo + sumoproject.dll) to a new location, then apply every parameter override the schematic specifies. Topology comes from the baseline; only parameter values change. Refuses to proceed when the schematic references variables not present in the baseline (unless strict=false is passed). NOTE: this tool keeps the .sumo zip byte-identical to the baseline and writes overrides into a sibling state.xml. If you want the overrides baked into the .sumo zip itself (rewriting parameters.txt + Influent tables + notes.rtf inside the archive), use build_native_sumo instead.
build_native_sumo	Native .sumo composer (Tier-1, baseline-DLL-reuse). Opens a verified baseline .sumo zip, rewrites the parameter-bearing internal members (parameters.txt, Influent*_Table*.tsv, notes.rtf, userscript.txt) from the supplied schematic, and writes a NEW zip that SUMO24 can open natively. Keeps the compiled sumoproject.dll byte-perfect and preserves every DLL-bound member. The plant topology is fixed by the baseline DLL; only parameter values change. Strict mode (default) refuses to build if any schematic unit lacks a baseline target — pass unit_mapping={schematic_id: baseline_id, ...} to override the heuristic class-based matcher. If the DTT bridge is live, the tool additionally tries load_model on the produced .sumo and reports the outcome in verify_load.
topology_match_report	Preview how a schematic maps onto a baseline .sumo BEFORE running build_native_sumo. Returns the proposed {schematic_id: baseline_id} mapping, the list of unmapped schematic units, the unused baseline units, and any class-mismatch warnings. Cheap to call — does not write any files.
validate_sumoslang	Lint a SumoSlang text for the most common breakages: brace balance, missing kineticmodel line, leftover TODO/FIXME markers. Use to pre-flight a generated plant.sumoslang before pasting into SUMO24.

verify_dll_matches_schematic	Inspect a sumoproject.dll's exported symbols and check that every sumo_variable referenced by the schematic exists in the DLL. Catches the common error of copying a DLL from one plant into another plant's directory — the file copy succeeds but the DLL has no idea what OxidationDitch1_T1 means and the project will load empty. Provide schematic_html or schematic_json for a full match; omit them to just list the DLL's exported Sumo__Plant__* symbols.
-------------------------------------	---

Group EE — Modelling Pipeline

19 tools

Tool	Description
next_step_for_project	Inspect a project directory and tell the user exactly what to do next. Looks for the .sumo stub, the build-pack, the DLL, state.xml, and an overrides-applied marker, then returns a single recommended next action. Use this any time the workflow feels ambiguous — it collapses the 5-step BUILD_INSTRUCTIONS.md into one guided call.
pipeline_init	Scaffold a new WWTP project directory with project.yaml manifest, HTML schematic template, and Excel data template. Required first call for any new plant. Sets up the 8-stage gated build pipeline.
pipeline_status	Return the complete pipeline state: current stage, status of every stage, stale detection (re-hashes artefacts on disk), and the exact next tool to call. Primary navigation tool for the pipeline.
pipeline_advance	Run the current stage validator and advance if it passes. Hash-chains the artefact and marks downstream stages stale on hash change. Use skip=True with a reason for optional stages 5 and 6.
pipeline_revert	Roll back the project to the state at the end of to_stage using the latest matching snapshot in <project_dir>/snapshots/.
pipeline_describe	Generate a human-readable Markdown report of the pipeline state: stage status, engineering values, provenance log, snapshots.
stage1_open_schematic	Stage 1 entry: return the HTML schematic path for this project.
stage1_validate_schematic	Stage 1 validator: parse HTML schematic, check unit types against registry, stream references, topology (influent+effluent required), orphan units, recycle hydraulics. Called automatically by pipeline_advance.
stage2_open_data	Stage 2 entry: ensure the Excel data template exists and pre-seed Treatment_Processes and Connections sheets from the Stage 1 schematic. Returns the Excel path.
stage2_validate_parameters	Stage 2 validator: validate_wwtp_template + cross-check unit names and connections against the Stage 1 schematic. Fails on any name mismatch.
stage3_run_engineering_checks	Stage 3: compute and validate SRT, HRT, F:M, AOR, OTR/AOR, COD mass balance, residual alkalinity, Takacs settler SOR/SLR. Reads from the Excel template. Writes engineering_checks.json.
stage4_define_static_inputs	Stage 4: resolve influent fractionation into ASM2d state-variable initial values. Writes static_inputs.json and static_inputs.scs.
stage4_validate_static_inputs	Stage 4 validator: check COD/N/P fraction sums, physical ranges, DO setpoints by reactor type, MLSS range, Q_RAS range, SRT cross-check.

stage5_generate_dynamic_inputs	Stage 5 (optional): produce a dynamic input TSV. profile_type: constant from_excel diurnal storm seasonal_temp. Output: <project_dir>/dynamic_inputs/<name>.tsv
stage5_validate_dynamic_inputs	Stage 5 validator: check dynamic_inputs/*.tsv files for uniform columns, monotonic time, physical value ranges, duration vs Sumo__StopTime.
stage6_add_controller	Stage 6 (optional): register a controller in controllers.json. Supported kinds: DO SRT NH4 NO3 FlowSplitter.
stage6_validate_controllers	Stage 6 validator: check controller unit references, setpoint ranges, no conflicting actuators.
stage7_build	Stage 7: assemble the build pack at <project_dir>/<plant_name>.sumo-pack/. Contents: plant.sumoslang, apply_parameters.scs, static_inputs.scs, dynamic_inputs/, controllers.scs (if not skipped), .sumo stub, BUILD_INSTRUCTIONS.md, schematic.json, engineering_checks.json.
stage8_verify	Stage 8 (post-SUMO-compile): verify DLL matches schematic, run trial steady-state, validate post-simulation mass balance. Call after compiling in SUMO24 GUI.

Group FF — DTT Edit & Transactions

5 tools

Tool	Description
begin_edit_transaction	Snapshot the current state.xml so a series of DTT edits can be rolled back if anything goes wrong. Returns a tx_id. Pair with commit_edit_transaction or rollback_edit_transaction.
commit_edit_transaction	Mark an edit transaction successful and (by default) discard its snapshot. Keep_snapshot=True retains the snapshot as a manual restore point.
rollback_edit_transaction	Restore the state.xml saved by begin_edit_transaction and re-initialise the in-memory model so DTT and disk agree. Backs the rolled-back state.xml aside as preRollback_*.bak.
apply_dtt_command_batch	Dispatch a vetted list of DTT command strings against the loaded model. Default mode rejects shell-injection tokens and any non-`set`/non-DTT verb; allow_raw=True bypasses vetting for power users. dry_run returns the planned commands without execution.
diff_inmemory_vs_disk	Compare the live in-memory model (units + streams) against the manifest of a .sumo on disk. Reports what needs save_model / reload to bring them into sync.

Group GG — SUMO GUI Automation

21 tools

Tool	Description
launch_sumo_gui	Launch (or attach to) the SUMO24 desktop GUI via pywinauto. Reads the executable path and window title regex from sumo_gui_layout.json. Returns the window bounding box on success. If SUMO is already running this just attaches.
gui_focus_sumo	Bring the SUMO24 main window to the foreground and return its rect. Call before any click/drag so the window is the active target.

gui_get_window_rect	Return the current SUMO24 main-window bounding box (screen coords). Use when you need to translate absolute screen positions to SUMO client coords.
gui_screenshot	Capture the SUMO24 main window to a PNG in output_dir. Useful for confirming canvas state after an edit or for hand-calibrating layout.canvas / layout.palette coordinates. Requires Pillow.
gui_switch_tab	Switch the SUMO ribbon tab. tab ∈ {Home, Configure, Models, Tools, Inputs, Outputs, Simulate}. Uses layout.tabs[tab] coords if present, otherwise the default SUMO24 1920-wide ribbon layout.
gui_click_at	Click at SUMO client (x, y). button ∈ {left, right, middle}; clicks ∈ {1, 2} (2 = double-click). Optional modifiers is a pywinauto send_keys held-key prefix (e.g. '+' for Shift, '^' for Ctrl, '%' for Alt).
gui_drag_from_to	Drag from SUMO client (x1, y1) to (x2, y2). Used for dropping palette icons onto the canvas, drawing connections between unit ports, and moving units.
gui_send_keys	Send a pywinauto key sequence to the focused SUMO window. Examples: '^s' (Ctrl+S), '{ENTER}', '{TAB 3}', '{F5}'.
gui_type_text	Type literal text into the focused SUMO field. Handles spaces and special characters that send_keys would otherwise treat as escape codes.
gui_close_dialog	Close the foreground dialog. action ∈ {ok (Enter), cancel (Escape), apply (Alt+A)}.
gui_list_palette_items	Return the current palette calibration table — which unit_types have (x, y) recorded and which are still uncalibrated.
gui_capture_palette_template	Capture a w×h rectangle centred at SUMO client (x, y) and save it as the palette template PNG for unit_type (PY/gui_templates/<unit_type>.png). Use once while the icon is clearly rendered; gui_drop_unit_by_template then finds the icon by appearance even if the palette reflows.
gui_calibrate_palette_item	Record the palette-icon (x, y) for unit_type into sumo_gui_layout.json. Use gui_screenshot first, read the icon centre with any image viewer, then call this once per process-unit type. Required before gui_drop_unit_at can use the icon.
gui_find_icon_by_template	Locate the palette icon for unit_type on screen via OpenCV TM_CCOEFF_NORMED. Returns centre point in SUMO client coords and match score. Optional search_bbox = [left, top, right, bottom] (client coords) restricts the search region.
gui_add_unit_to_canvas	Drag a treatment-unit icon from the SUMO24 palette to (x, y) on the canvas via the GUI. unit_type must be one of the keys in layout.palette (e.g. AerationTank, PrimaryClarifier). If x / y are omitted the layout's default_drop is used. Call launch_sumo_gui first.
gui_drop_unit_at	Drag the calibrated palette icon for unit_type onto the canvas at (x, y). Both source and destination are in SUMO client coords. Requires gui_calibrate_palette_item to have run for unit_type. Unlike gui_add_unit_to_canvas, this tool does NOT rely on UIA name lookup (which fails for SUMO's owner-drawn palette icons).
gui_drop_unit_by_template	Find the palette icon for unit_type via template matching, then drag it to canvas (x, y). The robust path for icons that hardcoded coords can't reliably grab — works regardless of palette reflow, panel-edge hit-test quirks, or window resize. Requires gui_capture_palette_template to have run once for unit_type.
gui_connect_units	Wire one unit to another by dragging from the outlet port of the source unit to the inlet port of the destination unit. Both coordinate pairs are in SUMO client pixels.

gui_open_unit_properties	Double-click a canvas unit at (x, y) to open its properties dialog. Returns the list of visible top-level window titles so the caller can confirm the dialog appeared.
gui_set_param_in_dialog	In the currently-open SUMO dialog, find the Edit control next to `label` (e.g. 'Volume', 'Fill ratio') and set it to `value`. Optionally press Enter after typing. Pass dialog_title_regex to target a specific dialog if the foreground window is wrong.
gui_save_sumo	Save the current SUMO24 GUI project as a .sumo file at target_path. Sends Ctrl+S (or layout.shortcut), waits for the Save dialog, types the path, and presses Enter. Returns ok=true only if the file exists after the dialog closes.

Group HH — SumoSlang Unit Authoring

24 tools

Tool	Description
hh_describe_process_unit	Prose front door: turn a natural-language description into a draft unit_spec. It infers STRUCTURE (ports, phases, species, process names) and NEVER a magnitude - every number comes back as the REQUIRED sentinel. Refuses to guess Type A/B routing.
hh_compose_unit_spec	Structured back door: build a unit_spec from explicit fields, with no NL inference. A value supplied without a provenance is REFUSED, not defaulted.
hh_read_unit_spec	Render a stored unit_spec for human review, ordered by danger: values still owed first.
hh_edit_unit_spec	Apply a structured edit at a named path (e.g. parameters[0].default.value). Setting a magnitude requires a provenance in the same call, and ANY edit clears acceptance.
hh_review_unit_spec	The review gate (rung 1 render): REQUIRED values, then unverified/self-asserted values, then structure and the Gujer matrix.
hh_accept_unit_spec	Record explicit, timestamped human acceptance with per-value confirmation. Emission refuses without it. Bulk confirmation is recorded and costs an explicit opt-in later.
hh_lint_unit_spec	Rung 1: enforces the REQUIRED sentinel, corroborated provenance, hedge detection and routing/composite refusal. Schema validation CANNOT do this.
hh_check_dimensions	Rung 1.5: dimensional consistency of units and expressions. Acceptance is ZERO FALSE POSITIVES, not a clean corpus - the shipped library contains 156 real vendor label bugs.
hh_check_continuity	Rung 2: per-process elemental continuity (COD/C/N/P/...) plus composition self-consistency. Gate is relative; the vendor 1E-15 is reported, not used as the gate.
hh_diff_model_base	Rung 2.5 (Type B only): semantic diff of a forked model base against the vendor original, keyed by symbol. Returns the additive-only gate verdict.
hh_compile_smt	Rung 3: SMT24.exe. Success is the ABSENCE of [ERROR] in the invocation-CWD smtlog plus a parseable XML - never the exit code. -nosort is refused outside diagnostics.
hh_compile_slcompiler	Rung 4: slcompiler.exe. Success requires the link-success line AND a DLL that verifies as PE32+ x86-64 - never the exit code.

hh_run_tracer_test	Rung 5: conservative tracer (5a) and degenerate equivalence (5b). Asserts on ROW COUNT and callback content, never the returned job id. Refuses -nosort builds.
hh_emit_unit_workbook	Emit the unit workbook and folder from an ACCEPTED spec, using xlsxwriter. Refuses on an unaccepted spec or a failing rung 1.
hh_emit_icon	Emit the unit's .emf icon: a GDI+ placeholder, or a template copy behind an opt-in. Stamps icon=placeholder or icon=template-copy=<src>.
hh_emit_test_plant	Emit the plant class + instance that make a unit compilable (SMT takes a plant instance as its entry point, never a unit). Enforces invariant I7: a REACTIVE plant is refused unless the unit itself declares the gas-transfer and flocculation symbols.
hh_fork_model_base	Type B: fork a whole model base additively. A fork with no edits is a byte-identical copy; row insertion goes through Excel COM (openpyxl drops 32 OOXML parts). Runs the rung-2.5 additive-only gate on its own output.
hh_add_unit_to_calibrated_plant	Ticket 18: admit a Type B unit to a calibrated plant. Takes the additive-only gate result as an INPUT and refuses without it; disables the unit on failure (Q14 tooth 4).
hh_install_process_unit	The ONLY tool that writes into D:\SUMO24, and only into the empty user overlay at Dir\My Process Code - never the vendor tree. Opt-in required; every install writes a manifest and uninstall removes exactly what that manifest lists. action: install uninstall list preflight; also installs a verified model-base fork.
hh_check_environment	Pre-flight: SMT/slcompiler present, licence registry entry and file, DTT importable, and which rung modules loaded.
hh_run_ladder	Run rungs 1 -> 1.5 -> 2 in order against a spec, stopping at the first failure and returning the whole trace. Compile rungs need artefacts and are called individually.
hh_clean_build	Wipe a slug's derived build/ and artifacts/ directories. Always safe - both regenerate.
hh_snapshot_build	Tar-snapshot a slug's state under a label before a risky step.
hh_revert_build	Restore a slug to a labelled snapshot.

Pipeline Overview

WWTP Excel Template (Group AA)

generate_wwtp_template_xlsx → fill the workbook → validate_wwtp_template → build_model_from_template. Builds a SUMO24 model from scratch, with sizing, operational parameters, and influent characteristics applied via the DTT bridge. apply_template_auto_fixes corrects common structural issues before building.

HTML Schematic (Group BB)

Open wwtp_schematic_template.html in a browser, build the plant using Configuration mode + Connection mode (loops supported for RAS/IR), click "Save HTML (for MCP)", then import_schematic_from_html → compare_schematic_to_model → build_model_from_schematic. update_schematic_in_html writes a revised schematic back into the HTML. Use generate_sumoslang_from_schematic for stretch-goal topology generation.

Troubleshooting (Group CC)

When a .sumo file won't open or simulate, start with validate_sumo_environment, then diagnose_sumo_file or scan_sumo_directory. check_dll_companion confirms the sumoproject.dll is present; repair_sumo_file fixes the most common issues. diagnose_sumo_crash parses recent SUMO logs for known error signatures. list_sumo_diagnostics returns the full catalogue of remediations.

Schematic-to-SUMO Compiler (Group DD)

compile_schematic_to_sumo or (preferred one-call) build_sumo_from_html → attach_companion_dll → verify_sumo_file_against_schematic → read_sumo_manifest. build_native_sumo handles full native-format builds; build_sumo_pack wraps the project into a distributable archive. topology_match_report, validate_sumoslang, and verify_dll_matches_schematic are available for post-build verification.

Guided Modelling Pipeline (Group EE)

next_step_for_project picks the right entry point. Structured path: pipeline_init → stage1_open_schematic / stage1_validate_schematic → stage2_open_data / stage2_validate_parameters → stage3_run_engineering_checks → stage4_define_static_inputs / stage4_validate_static_inputs → stage5_generate_dynamic_inputs / stage5_validate_dynamic_inputs → stage6_add_controller / stage6_validate_controllers → stage7_build → stage8_verify. pipeline_status, pipeline_advance, pipeline_revert, and pipeline_describe manage progress through the pipeline.

SumoSlang Unit Authoring (Group HH)

Author a NEW process unit from a natural-language description, in seven verified steps. Describe:

hh_describe_process_unit (prose) or hh_compose_unit_spec (explicit fields) → hh_edit_unit_spec fills values → hh_review_unit_spec / hh_accept_unit_spec record explicit human acceptance. Verify, in order: hh_lint_unit_spec (rung 1) → hh_check_dimensions (1.5) → hh_check_continuity (2) → hh_diff_model_base (2.5, Type B only) → hh_compile_smt (3) → hh_compile_slcompiler (4) → hh_run_tracer_test (5). hh_run_ladder sequences rungs 1–2 and stops at the first failure. Emit: hh_emit_unit_workbook + hh_emit_icon + hh_emit_test_plant (SMT compiles a PLANT, never a bare unit). hh_fork_model_base handles Type B. hh_install_process_unit is the ONLY tool that writes into the SUMO installation, it needs an explicit opt-in, and it writes only into the user overlay. Two rules run through all of it: STRUCTURE MAY BE INFERRED, MAGNITUDES MAY NOT — no tool here writes a number nobody supplied; and NO TOOL TRUSTS AN EXIT CODE — SMT24 returns 0 on a reader error, slcompiler returns 0 while logging [ERROR], and the DTT scheduler returns a job id even when the model failed to load.

SUMO GUI Automation (Group GG)

For the parts of SUMO that have no headless equivalent. launch_sumo_gui → gui_focus_sumo → gui_screenshot to see the window state. Palette work: gui_list_palette_items, gui_capture_palette_template and gui_calibrate_palette_item build a template library; gui_find_icon_by_template and gui_drop_unit_by_template then place units reliably. gui_connect_units draws streams; gui_open_unit_properties + gui_set_param_in_dialog edit a unit; gui_save_sumo persists. Prefer the headless tools wherever they exist — GUI automation is positional and brittle by nature.

DTT Edit & Transactions (Group FF)

begin_edit_transaction (snapshots state.xml) → apply_dtt_command_batch (dispatches vetted executeCommand strings in bulk) → commit_edit_transaction (persists changes) or rollback_edit_transaction (restores snapshot).

`diff_inmemory_vs_disk` reports what needs `save_model` / `reload` to bring the live in-memory model into sync with a `.sumo` on disk.